

1. Introduction

This project implements a concurrent task dispatcher in Rust that simulates how an operating system schedules and executes tasks. The system generates a large number of tasks, places them into a shared queue, and assigns them to a fixed number of worker threads based on a scheduling policy.

The purpose of this project is to demonstrate understanding of concurrency concepts such as threads, shared memory, synchronization, scheduling, and performance measurement.

2. System Architecture

The system is composed of the following components:

1. Task Generator

The task generator creates 500 tasks using a random distribution. Each task contains:

- id – unique identifier
- arrival_time – simulated arrival
- kind – CPU-bound or IO-bound
- duration – simulated execution time
- created_at – timestamp for metrics

2. Shared Queue

All tasks are stored in a shared queue implemented using:

`Arc<Mutex<VecDeque<Task>>>`

- VecDeque provides efficient FIFO operations
- Mutex ensures only one thread accesses the queue at a time
- Arc allows multiple threads to share ownership safely

3. Worker Threads

The system uses a bounded worker pool of 4 threads. Each worker:

- Continuously pulls tasks from the queue
- Processes the task (simulated with sleep)

- Updates global metrics
- Stops when the queue is empty

4. Metrics Collector

A shared Metrics struct tracks system performance:

- total tasks completed
 - CPU tasks completed
 - IO tasks completed
 - total wait time
 - total turnaround time
-

3. Synchronization Strategy

The system uses `Arc<Mutex<...>>` to safely share data between threads.

Why Mutex?

The queue is a shared resource. Without synchronization, multiple threads could:

- remove the same task
- corrupt the queue state

The Mutex ensures:

- only one worker accesses the queue at a time
- safe task removal

Why Arc?

Rust requires shared ownership across threads. Arc allows:

- multiple threads to reference the same queue
- safe memory management without data races

Trade-off

- Simplicity vs performance
- Mutex introduces some contention, but keeps design simple and correct

4. Scheduling Policies

Two scheduling policies were implemented:

1. FIFO (First-In First-Out)

- Tasks are processed in arrival order
- Implemented using `pop_front()`

Advantages:

- Fair (tasks are processed in order)
- Simple implementation

Disadvantages:

- Can lead to long wait times if large tasks come first
-

2. Optimized (Shortest Job First - SJF)

- Worker selects the task with the smallest duration
- Implemented by scanning the queue and selecting the minimum

Advantages:

- Reduces average wait time
- Improves system responsiveness

Disadvantages:

- Less fair (long tasks may starve)
 - Slightly higher overhead due to scanning
-

5. Metrics and Instrumentation

The system collects the following metrics:

- **Total tasks completed**
- **CPU tasks completed**

- **IO tasks completed**
- **Makespan (total runtime)**
- **Average wait time**
- **Average turnaround time**

Definitions

- **Wait time:** time from task creation to start of execution
- **Turnaround time:** total time from creation to completion
- **Makespan:** total time to complete all tasks

These metrics allow comparison between scheduling policies.

6. Experiments and Results

Two experiments were conducted:

Experiment A: FIFO Scheduling

Configuration:

- 500 tasks
- Mixed CPU and IO workload
- FIFO scheduling

Observations:

- Fair task ordering
 - Higher average wait time
 - Longer tasks delay shorter ones
-

Experiment B: Optimized Scheduling (SJF)

Configuration:

- Same workload

- Shortest Job First scheduling

Observations:

- Lower average wait time
- Faster completion for short tasks
- Less fairness (long tasks delayed)

Comparison Summary

Metric	FIFO	Optimized (SJF)
Fairness	High	Lower
Avg Wait Time	Higher	Lower
Throughput	Moderate	Better

Conclusion:

The optimized scheduler improves performance but sacrifices fairness.

7. Challenges and Bugs

Bug Encountered

Initially, the scheduling logic was placed outside the worker threads. This caused:

- compilation errors
- incorrect behavior due to missing thread context

Fix

The scheduling logic was moved inside the worker loop, ensuring:

- proper access to the shared queue
 - correct synchronization
-

8. Potential Issues

Starvation

In the optimized policy (SJF), long tasks may wait indefinitely if shorter tasks keep arriving.

Lock Contention

Using a single Mutex may create contention if many threads try to access the queue simultaneously.

9. Conclusion

This project demonstrates a working concurrent task dispatcher using Rust. The system successfully models task scheduling, thread coordination, and performance measurement.

The comparison between FIFO and optimized scheduling shows the trade-off between fairness and efficiency, which is a key concept in operating systems.

10. Future Improvements

Possible enhancements include:

- multiple queues (CPU vs IO)
- priority-based scheduling
- aging to reduce starvation
- worker-specific queues
- dynamic worker allocation